

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

по дисциплине «Введение в нереляционные базы данных»

**Тема: «Система мониторинга микроклимата учебных корпусов СПбГЭТУ
«ЛЭТИ»»**

Студенты гр. 3388

Басик В. В.
Глебова В. С.
Раутио И. А.

Преподаватель

Заславский М. М.

Санкт-Петербург

2026

ЗАДАНИЕ НА ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ

Студенты: Басик В. В.

Глебова В. С.

Раутио И. А.

Группа 3388

Тема проекта: «Система мониторинга микроклимата учебных корпусов»

Исходные данные:

Разработка веб-приложения на FastAPI + InfluxDB 2.7 с сравнительным анализом моделей данных (InfluxDB vs PostgreSQL), реализация CRUD, ролевой модели, импорта/экспорта, визуализации.

Содержание пояснительной записки:

Введение, 1. Анализ требований и архитектуры, 2. Проектирование моделей данных, 3. Реализация и сравнительный анализ, Заключение, Список использованных источников, Приложения.

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Дата выдачи задания: 13.02.2026

Дата сдачи реферата: 27.05.2026

Дата защиты реферата: 27.05.2026

Студенты

Басик В. В.
Глебова В. С.
Раутио И. А.

Преподаватель

Заславский М. М.

АННОТАЦИЯ

В рамках курса разработана система мониторинга микроклимата учебных корпусов СПбГЭТУ «ЛЭТИ». Решение реализовано в виде веб-приложения на FastAPI с хранением данных в InfluxDB 2.7 и клиентом на Bootstrap 5 / Chart.js. Проведён сравнительный анализ нереляционной (InfluxDB) и реляционной (PostgreSQL) моделей данных по объёму хранения и производительности запросов. Экспериментально подтверждено, что InfluxDB обеспечивает двукратное сокращение дискового пространства на текущем объёме и до 18-кратного при накоплении годовых данных, тогда как PostgreSQL демонстрирует в 10–60 раз меньшую задержку чтения. Система поддерживает CRUD-операции, многокритериальную фильтрацию, серверную пагинацию, импорт/экспорт данных и ролевую модель доступа. Найти исходный код и всю дополнительную информацию можно по ссылке: <https://github.com/moevm/nsql1h26-climate>

SUMMARY

Within the course, a microclimate monitoring system for educational buildings of Saint Petersburg Electrotechnical University "LETI" was developed. The solution is implemented as a web application with a FastAPI backend, InfluxDB 2.7 for time-series data storage, and a browser-based client built with Bootstrap 5 and Chart.js. A comparative analysis of non-relational (InfluxDB) and relational (PostgreSQL) data models was conducted with respect to storage efficiency and query performance. Experimental results confirm that InfluxDB reduces disk usage by a factor of 2 for the current dataset and up to 18× for projected yearly accumulation, whereas PostgreSQL demonstrates 10–60× lower read latency. The system supports full CRUD operations, multi-criteria filtering, server-side pagination, data import/export in JSON and CSV formats, and role-based access control (admin/viewer). Source code and supplementary materials are available at: <https://github.com/moevm/nsql1h26-climate>

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1.1. Актуальность решаемой проблемы	5
1.2. Постановка задачи	5
1.3. Предлагаемое решение	6
1.4. Качественные требования к решению	7
1. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ	9
1.1. Макет пользовательского интерфейса	9
1.2. Импорт данных	9
1.2.1. Массовый импорт из файла	10
1.2.2. Ручной ввод данных	10
1.3. Представление данных	10
1.4. Анализ данных	11
1.5. Экспорт данных	11
1.6. Вывод о преобладающем типе операций	12
2. МОДЕЛЬ ДАННЫХ	13
2.1. Нереляционная модель данных (InfluxDB 2.7)	13
2.1.1. Графическое представление	13
2.1.2. Описание назначений коллекций, типов данных и сущностей	14
2.1.3. Оценка удельного объема информации, хранимой в модели	15
2.1.4. Запросы к модели, с помощью которых реализуются сценарии использования	17
2.2. Аналог модели данных для SQL СУБД (PostgreSQL 15)	19
2.2.1. Графическое представление	19
2.2.2. Описание таблиц	20
2.2.3. Оценка удельного объема информации, хранимой в модели	21
2.2.4. Запросы к модели, с помощью которых реализуются сценарии использования	22
2.3. Сравнение моделей	24
2.3.1. Удельный объем информации	24
2.3.2. Запросы по отдельным юзкейсам	25
2.3.3. Вывод — что лучше, SQL или NoSQL модель	27
3. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ	29
3.1. Краткое описание архитектуры	29
3.2. Используемые технологии	29
3.3. Снимки экрана приложения	30
ЗАКЛЮЧЕНИЕ	33
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	35
ПРИЛОЖЕНИЕ А. ДОКУМЕНТАЦИЯ ПО СБОРКЕ И РАЗВЕРТЫВАНИЮ ПРИЛОЖЕНИЯ	36
ПРИЛОЖЕНИЕ Б. ИНСТРУКЦИЯ ДЛЯ ПОЛЬЗОВАТЕЛЯ	37

ВВЕДЕНИЕ

Целью данной работы является разработка веб-приложения для непрерывного мониторинга микроклимата учебных и административных корпусов СПбГЭТУ «ЛЭТИ», а также сравнительный анализ эффективности нереляционной (InfluxDB) и реляционной (PostgreSQL) СУБД при хранении и обработке временных рядов IoT-данных.

1.1. Актуальность решаемой проблемы

Качество воздуха и микроклимат в учебных помещениях непосредственно влияют на работоспособность, концентрацию внимания и здоровье студентов и преподавателей. Повышенная концентрация CO₂ (свыше 1000 ppm) снижает когнитивные способности и вызывает быструю утомляемость; отклонения температуры и влажности от нормативов СанПиН нарушают тепловой комфорт и способствуют распространению респираторных заболеваний. При этом университетские корпуса содержат сотни помещений, ручной контроль параметров в которых трудоёмок, субъективен и не позволяет оперативно реагировать на отклонения. Автоматизированная система мониторинга обеспечивает сбор, хранение и визуализацию показателей в режиме реального времени, что даёт возможность принимать обоснованные управленческие решения по вентиляции и отоплению.

Дополнительную научную и практическую актуальность задаёт необходимость выбора оптимального хранилища для IoT-нагрузок. Сравнительное исследование специализированной time-series СУБД и классической реляционной модели позволяет сформировать практические рекомендации по архитектуре аналогичных систем, учитывая требования к объёму данных, скорости запросов и масштабируемости.

1.2. Постановка задачи

Для достижения поставленной цели требуется разработать систему, обеспечивающую выполнение следующих требований:

- хранение иерархической структуры объектов: корпус → этаж → помещение → датчик;
- непрерывный приём и сохранение показаний датчиков (температура, влажность, концентрация CO₂);
- визуализация текущих значений и динамики показателей за произвольный период;
- многокритериальный поиск и фильтрация данных по любому сочетанию атрибутов;
- анализ распределения объектов по заданным параметрам с построением настраиваемых диаграмм;
- импорт и экспорт данных в машиночитаемых форматах (JSON, CSV);
- разграничение прав доступа: роль администратора (полный доступ) и наблюдателя (только чтение);
- проведение сравнительного анализа нереляционной (InfluxDB 2.7) и реляционной (PostgreSQL 15) моделей данных по критериям объёма хранения, сложности запросов и производительности.

1.3. Предлагаемое решение

Система реализована в виде клиент-серверного веб-приложения, развёртываемого через Docker Compose. Архитектура включает два основных контейнера: бэкенд на FastAPI (сервер Uvicorn) и базу данных InfluxDB 2.7. Браузерный клиент представляет собой одностраничное приложение (SPA), написанное на нативном JavaScript с использованием Bootstrap 5 для вёрстки и Chart.js для визуализации; статические файлы отдаются бэкендом.

Выбор InfluxDB в качестве основной СУБД обусловлен её специализацией под временные ряды: алгоритм TSM-сжатия обеспечивает компактное хранение плавно изменяющихся метрик, а теговая модель позволяет эффективно индексировать и фильтровать данные по измерениям (корпус, этаж, помещение, тип метрики) без создания дополнительных

индексов. Метаданные объектов (корпуса, этажи, помещения, датчики) хранятся в том же бакете в виде JSON-полей с версионированием и флагом мягкого удаления, что устраняет необходимость в отдельной реляционной БД для справочников.

1.4. Качественные требования к решению

Для обеспечения надёжной, безопасной и удобной эксплуатации системы были сформулированы следующие качественные требования:

1. **Доступность.** Веб-интерфейс должен корректно функционировать в любом современном браузере без необходимости установки дополнительного программного обеспечения или плагинов. *Критерий выполнения:* главная страница открывается без ошибок кэширования, зависимостей или блокировок контента.

2. **Производительность.** Время полной загрузки основных страниц не должно превышать 2 секунд при текущем объёме данных (≈ 345 датчиков и $\sim 17\,000$ записей показаний). *Критерий выполнения:* подтверждается инструментальным замером через панель разработчика браузера (DevTools $\rightarrow$ Network).

3. **Масштабируемость.** Архитектура системы должна допускать горизонтальное масштабирование ключевых компонентов, в частности хранилища InfluxDB и бэкенда. *Критерий выполнения:* возможность запуска нескольких реплик контейнеров через оркестраторы (Docker Swarm / Kubernetes) без модификации бизнес-логики приложения.

4. **Безопасность.** Доступ к REST API должен строго контролироваться механизмом JWT-аутентификации, а порты базы данных не должны публиковаться во внешнюю сеть. *Критерий выполнения:* запросы без валидного токена авторизации возвращают HTTP 401; порт 8086 доступен исключительно внутри внутренней Docker-сети.

5. **Корректность данных.** Все входные параметры запросов должны проходить строгую валидацию на уровне схем Pydantic v2 до попадания в

бизнес-логику. *Критерий выполнения:* некорректные, неполные или типизированные неверно JSON-запросы автоматически отклоняются с кодом ответа HTTP 422 и детальным описанием ошибок в теле ответа.

6. **Воспроизводимость.** Процесс развёртывания системы должен выполняться одной командой на любой чистой машине с установленным Docker. *Критерий выполнения:* успешный запуск `docker compose up --build` без дополнительных ручных настроек окружения, переменных или предустановленных пакетов.

7. **Наблюдаемость.** Система должна обеспечивать фоновую генерацию синтетических показаний для отладки визуализации и демонстрации работы интерфейса в отсутствие реальных датчиков. *Критерий выполнения:* автоматическое появление новых записей в бакете `climate_readings` с заданной периодичностью (каждые 30 минут) без вмешательства пользователя.

1. СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

1.1. Макет пользовательского интерфейса

Интерфейс системы представляет собой одностраничное веб-приложение (SPA) с фиксированным боковым меню навигации и основной рабочей областью. Макет включает следующие структурные блоки:

- Панель навигации (слева): разделы «Дашборд», «Корпуса», «Помещения», «Датчики», «Статистика», «Импорт/Экспорт». Доступность пунктов зависит от роли пользователя (администратор/наблюдатель).
- Верхняя панель: информация о текущей роли, кнопка выхода, индикатор состояния соединения с API.
- Основная область: динамически подгружаемые таблицы, карточки сводных метрик, формы модальных окон и интерактивные графики.
- Адаптивность: интерфейс корректно масштабируется на экранах от 1366×768 пикселей, таблицы поддерживают горизонтальную прокрутку на мобильных устройствах.

Графическое представление макета приведено на рисунке 1. Файл PNG размещен в корне репозитория.



Рисунок 1 — Макет пользовательского интерфейса системы

1.2. Импорт данных

Система поддерживает два способа добавления данных: массовую загрузку из файла и ручной ввод через интерфейс.

1.2.1. Массовый импорт из файла

Администратор переходит в раздел «Импорт/Экспорт», выбирает тип сущности (корпуса, этажи, помещения, датчики или показания) и загружает файл в формате JSON или CSV. Система выполняет следующие действия:

1. Валидирует структуру файла на соответствие схеме Pydantic v2.
2. Проверяет уникальность идентификаторов и корректность ссылочных связей (например, `building_id` должен существовать в таблице корпусов).
3. При успешной проверке записывает данные в InfluxDB батчами по 500 записей для минимизации нагрузки на сеть.
4. Возвращает пользователю отчёт: «Успешно импортировано: X записей, ошибок: Y». При обнаружении критических ошибок импорт прерывается, транзакция откатывается, пользователю выводится детальное описание ошибки с указанием строки файла.

1.2.2. Ручной ввод данных

На странице соответствующей сущности (например, «Датчики») администратор нажимает кнопку «+ Добавить». Открывается модальное окно с формой, содержащей обязательные и опциональные поля. После заполнения полей пользователь нажимает «Сохранить». Система отправляет POST-запрос к REST API, выполняет валидацию, создаёт запись в БД и немедленно отображает новую строку в таблице без перезагрузки страницы.

1.3. Представление данных

Данные отображаются в виде интерактивных таблиц и сводных карточек. Для работы с отображением пользователь выполняет следующие действия:

- Навигация: выбирает раздел в боковом меню. Система загружает первую страницу данных (по умолчанию 20 записей на страницу).
- Поиск и фильтрация: в каждой таблице предусмотрены отдельные поля ввода для каждого атрибута (название, адрес, тип метрики, статус, дата

создания/изменения). Поиск по текстовым полям регистронезависимый и выполняется по подстроке. Пользователь вводит критерии и нажимает «Применить». Запрос формируется на сервере, данные возвращаются с серверной пагинацией.

- Визуализация: на дашборде автоматически отображаются средние значения температуры, влажности и CO₂ по корпусам. При клике на строку таблицы открывается детальный просмотр связанной сущности (например, список этажей внутри корпуса или последние показания датчика).

1.4. Анализ данных

Для проведения аналитики пользователь переходит в раздел «Статистика». Система предоставляет два инструмента:

1. График временного ряда: пользователь выбирает корпус, тип метрики, временной диапазон (например, последние 7 дней) и интервал агрегации (1 ч / 2 ч / 6 ч / 12 ч / 24 ч). После нажатия «Построить» система выполняет агрегирующий запрос к InfluxDB (aggregateWindow) и отображает линейный график в Chart.js.

2. Анализ распределения: пользователь задаёт тип объектов (датчики/помещения), многокритериальный фильтр (корпус, статус, тип помещения), атрибут для оси X (например, building_id) и атрибут группировки для оси Y (например, metric_type). Система формирует сгруппированную столбчатую диаграмму, показывающую количественное распределение объектов по выбранным параметрам. Пользователь может экспортировать данные графика в PNG прямо из интерфейса.

1.5. Экспорт данных

Для получения копии данных в машиночитаемом формате администратор переходит в раздел «Импорт/Экспорт» и выбирает тип сущности. Система формирует файл и инициирует его скачивание в браузере:

- Метаданные (корпуса, этажи, помещения, датчики) экспортируются в формате JSON с полным набором полей и историей изменений.

- Показания датчиков (`climate_readings`) экспортируются в формате CSV с колонками: `timestamp`, `sensor_id`, `building_id`, `metric_type`, `value`, `status`. Экспорт выполняется одним действием без необходимости ручной выборки данных. Файл формируется на сервере, после чего отдаётся клиенту с заголовком `Content-Disposition: attachment`.

1.6. Вывод о преобладающем типе операций

Разработанная система является читательски-интенсивной (*read-heavy*). Фоновый генератор синтетических данных и редкие ручные операции администратора формируют непрерывный, но низкоинтенсивный поток записи (≈ 1 запись в 30 минут на каждый из ~ 345 активных датчиков). В то же время каждый пользовательский сеанс включает многократные операции чтения: загрузка дашборда, применение фильтров, построение графиков, просмотр детальных карточек и экспорт отчётов. Соотношение операций оценивается как $\approx 85\text{--}90\%$ чтения к $10\text{--}15\%$ записи. Это обосновывает приоритетную оптимизацию запросов на чтение (индексация по тегам, кэширование агрегированных рядов, серверная пагинация) и выбор СУБД, эффективно работающей с частыми выборками из больших временных рядов.

2. МОДЕЛЬ ДАННЫХ

2.1. Нереляционная модель данных (InfluxDB 2.7)

2.1.1. Графическое представление

Модель данных в InfluxDB построена на основе 6 measurements (измерений), из которых 4 хранят метаданные иерархической структуры объектов, а 2 — временные ряды показаний и событий. Визуальное представление модели приведено на рисунке 2.

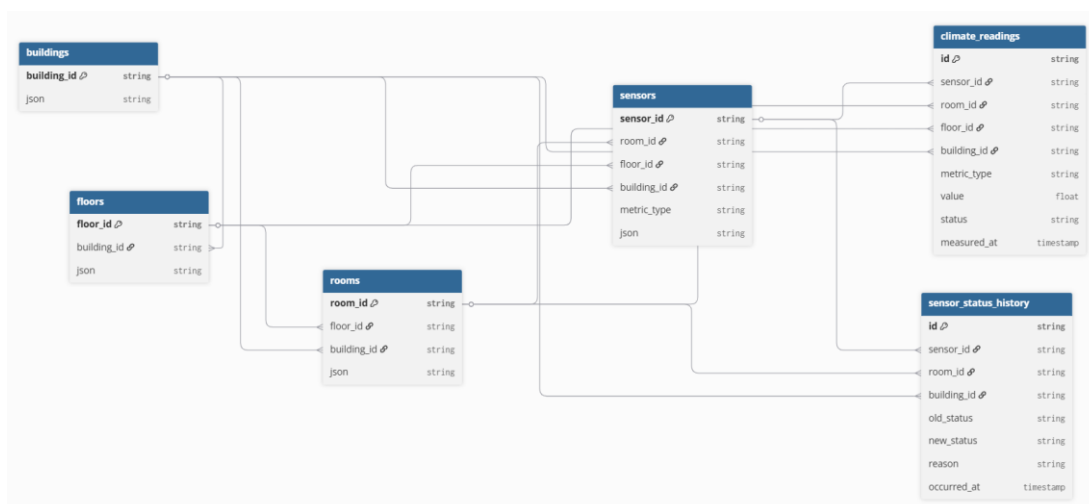


Рисунок 2 — Логическая модель данных системы (InfluxDB 2.7)

На схеме отображены следующие сущности:

- `buildings`, `floors`, `rooms`, `sensors` — измерения, хранящие метаданные об объектах. Каждое измерение содержит уникальный идентификатор в виде тега и `json`-поле с расширенными атрибутами (название, статус, даты создания/редактирования, история изменений).
- `climate_readings` — основное измерение временных рядов, содержащее показания датчиков (температура, влажность, концентрация CO₂). Связи с другими сущностями реализованы через теги (`sensor_id`, `room_id`, `floor_id`, `building_id`, `metric_type`), что обеспечивает быструю фильтрацию и группировку без явных операций JOIN.
- `sensor_status_history` — измерение для аудита, фиксирующее изменения статуса датчиков.

В отличие от реляционных СУБД, InfluxDB не использует строгие ограничения внешних ключей (FOREIGN KEY). Связи на схеме носят логический характер и реализованы через согласованные значения тегов, что соответствует архитектуре time-series баз данных и обеспечивает высокую производительность при записи и чтении потоковых данных.

2.1.2. Описание назначений коллекций, типов данных и сущностей

Модель данных в InfluxDB 2.7 базируется на концепции измерений (measurements), каждое из которых представляет собой логическую группу записей с общей структурой тегов и полей. В разработанной системе выделено шесть измерений, отражающих иерархическую организацию объектов мониторинга и потоковые данные с датчиков. Структура, типы данных и назначение измерений приведены в таблице 1.

Таблица 1 — Структура и назначение измерений InfluxDB

Measurement	Теги (T) / Поля (F)	Тип данных	Назначение
buildings	T: building_id F: json	string / string	Метаданные корпусов: название, адрес, количество этажей, статус, даты создания/изменения
floors	T: floor_id, building_id F: json	string, string / string	Метаданные этажей: номер этажа, статус, даты
rooms	T: room_id, floor_id, building_id F: json	string, string, string / string	Метаданные помещений: номер, тип, площадь, комментарий, статус

sensors	T: sensor_id, room_id, floor_id, building_id, metric_type F: json	string × 5 / string	Метаданные датчиков: модель, статус, даты, история изменений
climate_readings	T: sensor_id, room_id, floor_id, building_id, metric_type F: value, status	string × 5 / float, string	Временной ряд показаний: температура, влажность, CO ₂ с временной меткой
sensor_status_history	T: sensor_id, room_id, building_id F: old_status, new_status, reason	string × 3 / string × 3	История смены статусов датчиков для аудита и отладки

2.1.3. Оценка удельного объёма информации, хранимой в модели

Для расчёта объёма хранимых данных введём следующие переменные:

- **N_b** — количество корпусов (в текущей реализации N_b = 8);
- **N_f** — количество этажей (N_f = 27);
- **N_r** — количество помещений (N_r = 115);
- **N_s** — количество датчиков (N_s = 345);
- **N_c** — количество показаний (N_c = 16 905 на момент анализа);
- **N_h** — количество записей истории статусов (N_h = 464).

Общий объём данных рассчитывается по формуле:

$$V_{total} = N_b \times V_b + N_f \times V_f + N_r \times V_r + N_s \times V_s + N_c \times V_c + N_h \times V_h$$

где V_x — средний размер одной записи соответствующего measurement (в байтах).

Зависимость объёма от количества объектов для каждого измерения:

Measurement	Формула зависимости	Коэффициент (байт/запись)
buildings	$V = N_b \times 203$	203
floors	$V = N_f \times 176$	176
rooms	$V = N_r \times 291$	291
sensors	$V = N_s \times 341$	341
climate_readings	$V = N_c \times 11$ (TSM-сжатие)	11
sensor_status_history	$V = N_h \times 174$	174

Расчёт объёма при текущих значениях ($N_b = 8$):

Measurement	Байт/запись	Объём при $N_b=8$
buildings	203	$203 \times 8 = 1\,624$ Б
floors	176	$176 \times 27 = 4\,752$ Б
rooms	291	$291 \times 115 = 33\,465$ Б
sensors	341	$341 \times 345 = 117\,645$ Б
climate_readings (TSM)	≈ 11	$11 \times 16\,905 = 185\,955$ Б
sensor_status_history	174	$174 \times 464 = 80\,736$ Б
Итого данные	—	$\approx 0,41$ МБ
TSI + shard overhead	—	$\approx 1,13$ МБ
Итого на диске	—	$\approx 1,54$ МБ (факт. 1,60 МБ)

Прогноз роста объёма:

При сохранении текущей интенсивности генерации данных (345 датчиков, 1 показание каждые 30 минут):

- Количество новых показаний в год: $N_{\text{с(год)}} = 345 \times 24 \times 2 \times 365 \approx 6\,039\,600$ записей

- Объём показаний через 1 год: $V_{\text{climate(год)}} \approx 6\,039\,600 \times 11 \approx 66,4$ МБ (с учётом TSM-сжатия)

- Прогнозируемый общий объём на диске через 1 год: $\approx 65\text{--}70$ МБ

Коэффициент избыточности модели:

Избыточность определяется как отношение фактического объёма на диске к объёму «чистых» данных:

$$R = V_{\text{disk}} / V_{\text{data}} \approx 1,60 \text{ МБ} / 0,41 \text{ МБ} \approx 3,9$$

Основной вклад в избыточность вносит TSI (Tag Search Index) — автоматический индекс по всем тегам, а также накладные расходы шардов InfluxDB. Это цена за высокую скорость фильтрации и группировки по тегам без явных индексов.

2.1.4. Запросы к модели, с помощью которых реализуются сценарии использования

Ниже приведены примеры запросов на языке Flux для реализации основных сценариев использования системы. Для каждого запроса указано соответствие сценарию, количество задействованных измерений и оценка асимптотической сложности.

Q1 — Список всех корпусов (сценарий «Представление данных»)

```
from(bucket: "climate")
  |> range(start: -36500d)
  |> filter(fn: (r) => r._measurement == "buildings")
  |> filter(fn: (r) => r._field == "json")
  |> group(columns: ["building_id"])
  |> last()
```

Запрос реализует сценарий просмотра списка корпусов. Выполняется одним обращением к измерению `buildings`. Сложность составляет $O(N_b)$, так как требуется сканирование всех записей с последующей группировкой по тегу `building_id`.

Q2 — Почасовой график температуры за 24 ч (сценарий «Анализ данных»)

```
from(bucket: "climate")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "climate_readings")
  |> filter(fn: (r) => r._field == "value")
  |> filter(fn: (r) => r.building_id == "bldg-1")
  |> filter(fn: (r) => r.metric_type == "temperature")
  |> aggregateWindow(every: 1h, fn: mean, createEmpty: false)
```

Запрос реализует сценарий построения графика показаний за период. Выполняется одним обращением к измерению `climate_readings`. Сложность составляет $O(K)$, где K — количество точек в заданном временном диапазоне; агрегация выполняется за линейное время благодаря оптимизации `aggregateWindow`.

Q3 — Датчики корпуса с последним показанием (сценарий «Представление данных»)

```
sensors = from(bucket: "climate")
  |> range(start: -36500d)
  |> filter(fn: (r) => r._measurement == "sensors")
  |> filter(fn: (r) => r.building_id == "bldg-1")
  |> group(columns: ["sensor_id"])
  |> last()

readings = from(bucket: "climate")
  |> range(start: -36500d)
  |> filter(fn: (r) => r._measurement == "climate_readings")
  |> filter(fn: (r) => r.building_id == "bldg-1")
  |> group(columns: ["sensor_id"])
  |> last()

join(tables: {s: sensors, r: readings}, on: ["sensor_id"])
```

Запрос реализует сценарий просмотра датчиков с актуальными показаниями. Требует двух отдельных выборок из измерений `sensors` и

climate_readings с последующим соединением через join. Сложность оценивается как $O(N_s \times \log T)$, где N_s — количество датчиков, T — количество показаний на датчик.

2.2. Аналог модели данных для SQL СУБД (PostgreSQL 15)

2.2.1. Графическое представление

Реляционная модель данных содержит шесть таблиц, связанных через внешние ключи: building, floor, room, sensor, climate_reading, sensor_status_event. Связи реализованы в соответствии с иерархией объектов: корпус → этаж → помещение → датчик → показания. Для ускорения запросов на таблице climate_reading созданы два B-tree индекса: по sensor_id и по составному ключу (building_id, metric_type, measured_at). Визуальное представление модели приведено на рисунке 3.

Для ускорения выполнения запросов на таблице climate_reading созданы два B-tree индекса: по полю sensor_id для быстрого поиска показаний конкретного датчика и составной индекс по полям (building_id, metric_type, measured_at) для оптимизации агрегирующих запросов с фильтрацией по корпусу, типу метрики и временному диапазону.

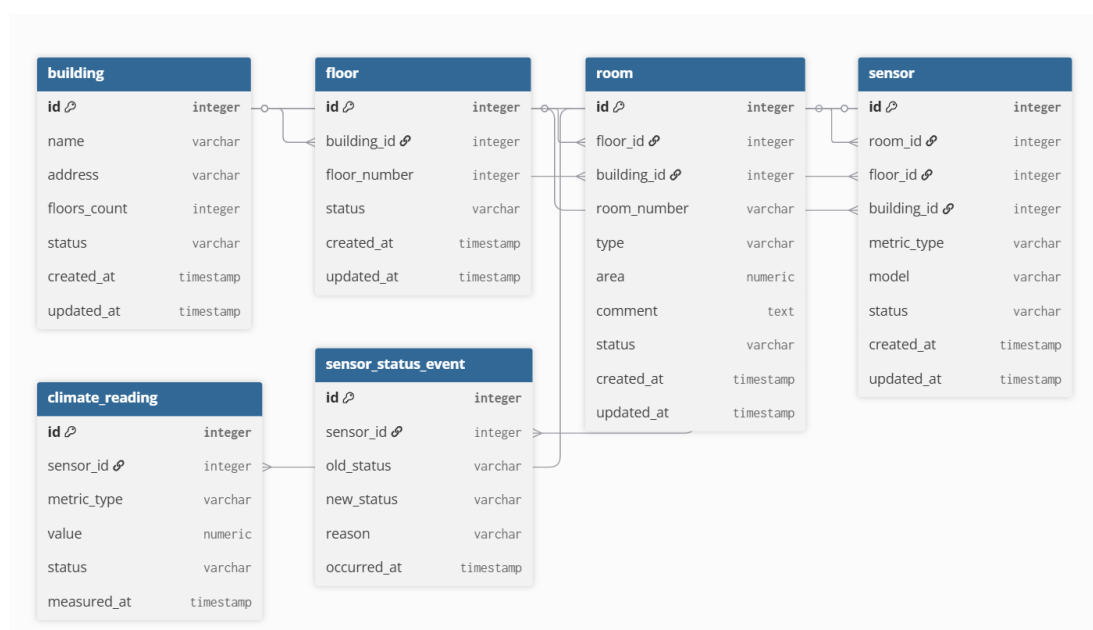


Рисунок 3 — ER-диаграмма реляционной модели PostgreSQL

2.2.2. Описание таблиц

Реляционная модель данных системы в PostgreSQL 15 включает шесть таблиц, каждая из которых соответствует определённому типу сущности иерархической структуры мониторинга. Структура таблиц, ключевые столбцы и размер строк приведены в таблице 2.

Таблица 2 — Структура таблиц реляционной модели (PostgreSQL)

Таблица	Ключевые столбцы	Размер строки, Б	Назначение
building	id (PK), name, address, floors_count, status	104	Метаданные корпусов: название, адрес, количество этажей, статус
floor	id (PK), building_id (FK), floor_number, status	66	Метаданные этажей: номер, связь с корпусом, статус
room	id (PK), floor_id (FK), building_id (FK), room_number, type, area, comment, status	110	Метаданные помещений: номер, тип, площадь, комментарий
sensor	id (PK), room_id (FK), floor_id (FK), building_id (FK), metric_type, model, status	119	Реестр датчиков: привязка к помещению, тип метрики, модель

climate_reading	id (PK, serial), sensor_id (FK), metric_type, value, status, measured_at	201 (с индексами)	Временной ряд показаний: температура, влажность, CO ₂
sensor_status_event	id (PK, serial), sensor_id (FK), old_status, new_status, reason, occurred_at	141	Аудит изменений статуса датчиков для отладки

2.2.3. Оценка удельного объёма информации, хранимой в модели

Зависимость объёма от количества объектов для каждой таблицы:

Measurement	Формула зависимости	Коэффициент (байт/запись)
buildings	$V = N_b \times 104$	104
floors	$V = N_f \times 66$	66
rooms	$V = N_r \times 110$	110
sensors	$V = N_s \times 119$	119
climate_readings	$V = N_c \times 201$	201
sensor_status_history	$V = N_h \times 141$	141

Расчёт объёма при текущих значениях ($N_b = 8$):

Таблица	Строк при $N_b=8$	Байт/строка	Итого
building	8	104	832 Б
floor	27	66	1 782 Б
room	115	110	12 650 Б
sensor	345	119	41 055 Б
climate_reading	16 905	201	3 397 905 Б
sensor_status_event	464	141	65 424 Б

Итого	—	—	≈ 3,29 МБ (факт. 3,34 МБ)
-------	---	---	------------------------------

Прогноз роста объёма:

При сохранении текущей интенсивности генерации данных (345 датчиков, 1 показание каждые 30 минут):

- Количество новых показаний в год: $N_{с(год)} = 345 \times 24 \times 2 \times 365 \approx 6\,039\,600$ записей
- Объём показаний через 1 год: $V_{climate(год)} \approx 6\,039\,600 \times 201 \approx 1,16$ ГБ
- Прогнозируемый общий объём на диске через 1 год: $\approx 1,2$ ГБ

Коэффициент избыточности модели:

Избыточность определяется как отношение фактического объёма на диске к объёму «чистых» данных:

$$R = V_{disk} / V_{data} \approx 3,34 \text{ МБ} / 3,29 \text{ МБ} \approx 1,02$$

Низкая избыточность объясняется нормализованной структурой реляционной модели и эффективным хранением индексов B-tree. В отличие от InfluxDB, где теги автоматически индексируются в TSI с коэффициентом избыточности $\sim 3,9$, в PostgreSQL индексы создаются явно и только для часто используемых полей, что минимизирует накладные расходы.

2.2.4. Запросы к модели, с помощью которых реализуются сценарии использования

Ниже приведены примеры запросов на языке SQL для реализации основных сценариев использования системы в реляционной модели (PostgreSQL). Для каждого запроса указано соответствие сценарию, количество задействованных таблиц и оценка асимптотической сложности.

Q1 — Список всех корпусов (сценарий «Представление данных»)

```
SELECT id, name, address, floors_count, status, created_at,
updated_at
FROM building
ORDER BY name;
```

Запрос реализует сценарий просмотра списка корпусов. Выполняется одним обращением к таблице `building`. Сложность составляет $O(\log N_b)$ благодаря автоматическому использованию B-tree индекса по первичному ключу и оптимизированной сортировке.

Q2 — Почасовой график температуры за 24 ч (сценарий «Анализ данных»)

```
SELECT DATE_TRUNC('hour', measured_at) AS bucket,
       metric_type,
       ROUND(AVG(value)::numeric, 2) AS avg_value
FROM climate_reading
WHERE building_id = $1
      AND metric_type = $2
      AND measured_at >= NOW() - INTERVAL '24 hours'
GROUP BY bucket, metric_type
ORDER BY bucket;
```

Запрос реализует сценарий построения графика показаний за период. Выполняется одним обращением к таблице `climate_reading`. Сложность составляет $O(K \log K)$, где K — количество точек в заданном временном диапазоне; группировка и сортировка выполняются с использованием составного B-tree индекса по полям (`building_id`, `metric_type`, `measured_at`).

Q3 — Датчики корпуса с последним показанием (сценарий «Представление данных»)

```
SELECT s.id, s.metric_type, s.model, s.status,
       cr.value AS last_value, cr.measured_at
FROM sensor s
LEFT JOIN LATERAL (
    SELECT value, measured_at
    FROM climate_reading
    WHERE sensor_id = s.id
    ORDER BY measured_at DESC
    LIMIT 1
) cr ON true
WHERE s.building_id = $1
ORDER BY s.id;
```

Запрос реализует сценарий просмотра датчиков с актуальными показаниями. Использует конструкцию LATERAL JOIN для эффективного получения последнего показания каждого датчика без полного сканирования таблицы. Сложность оценивается как $O(N_s \times \log T)$, где N_s — количество датчиков в корпусе, T — количество показаний на датчик. Задействует две таблицы (sensor и climate_reading).

2.3. Сравнение моделей

2.3.1. Удельный объём информации

Для сравнения эффективности хранения данных в нереляционной (InfluxDB) и реляционной (PostgreSQL) моделях используются одинаковые входные параметры:

- N_b — количество корпусов (8);
- N_f — количество этажей (27);
- N_r — количество помещений (115);
- N_s — количество датчиков (345);
- N_c — количество показаний (16 905);
- N_h — количество записей истории статусов (464).

Общий объём данных для обеих моделей рассчитывается по единой формуле:

$$V_{total} = N_b \times V_b + N_f \times V_f + N_r \times V_r + N_s \times V_s + N_c \times V_c + N_h \times V_h$$

Сравнение коэффициентов и фактических объёмов приведено в таблице 3

Таблица 3 — Сравнение объёма хранения данных

Параметр	InfluxDB (NoSQL)	PostgreSQL (SQL)	Соотношение
Расчётный объём данных	0,41 МБ	3,29 МБ	PostgreSQL в 8× больше
Фактический объём на диске	1,60 МБ	3,34 МБ	PostgreSQL в 2,1× больше

Избыточность (данные)	$\approx 3,9$ (TSI + shards)	$\approx 1,02$ (B-tree)	InfluxDB имеет большие накладные расходы на индексы
Объём через 1 год (345 датчиков)	≈ 65 МБ	$\approx 1,2$ ГБ	PostgreSQL в ~18× больше

Анализ:

- InfluxDB демонстрирует значительное преимущество в эффективности хранения временных рядов благодаря алгоритму TSM-сжатия, который уменьшает объём плавно изменяющихся метрик в ~ 18 раз.
- Более высокий коэффициент избыточности InfluxDB ($\sim 3,9$) объясняется автоматическим индексированием всех тегов в TSI (Tag Search Index) и накладными расходами шардов — это цена за высокую скорость фильтрации без явных индексов.
- В PostgreSQL низкая избыточность ($\sim 1,02$) достигается за счёт нормализованной структуры и создания индексов только для часто используемых полей, однако отсутствие специализированного сжатия для временных рядов приводит к большему итоговому объёму.

Вывод по объёму: при прочих равных условиях данные в PostgreSQL занимают больший объём — в 2 раза больше на текущем наборе и до 18 раз больше при долгосрочном накоплении. Для систем IoT с непрерывным потоком показаний это критичный фактор.

2.3.2. Запросы по отдельным юзкейсам

Сравнение реализации ключевых сценариев использования в нереляционной (InfluxDB) и реляционной (PostgreSQL) моделях проведено на основе запросов, приведённых в разделах 2.1.4 и 2.2.4. Для каждого сценария оценено количество запросов в зависимости от числа объектов в базе данных,

число задействованных измерений/таблиц и асимптотическая сложность выполнения.

Юзкейс 1 — Просмотр списка корпусов

Сценарий реализуется одним запросом в обеих моделях независимо от количества объектов. Задействуется одно измерение (buildings) в InfluxDB и одна таблица (building) в PostgreSQL. Сложность в InfluxDB составляет $O(N_b)$ — требуется сканирование всех записей измерения с последующей группировкой по тегу building_id. В PostgreSQL сложность $O(\log N_b)$ благодаря использованию B-tree индекса по первичному ключу id. При увеличении числа корпусов в 10 раз время выполнения запроса в InfluxDB возрастёт линейно, тогда как в PostgreSQL — логарифмически.

Юзкейс 2 — Построение почасового графика температуры за 24 ч

Сценарий реализуется одним запросом в обеих моделях. Задействуется одно измерение (climate_readings) в InfluxDB и одна таблица (climate_reading) в PostgreSQL. Сложность в InfluxDB составляет $O(K)$, где K — количество точек в заданном временном диапазоне; агрегация выполняется за линейное время благодаря оптимизации функции aggregateWindow. В PostgreSQL сложность $O(K \log K)$ из-за необходимости выполнения группировки GROUP BY и сортировки результатов. При увеличении частоты опроса датчиков в 2 раза (удвоение K) время выполнения в InfluxDB возрастёт в 2 раза, в PostgreSQL — более чем в 2 раза из-за логарифмического множителя.

Юзкейс 3 — Просмотр датчиков корпуса с последним показанием

Сценарий реализуется двумя запросами с последующим join в InfluxDB и одним запросом с LATERAL JOIN в PostgreSQL. Задействуется два измерения (sensors, climate_readings) в InfluxDB и две таблицы (sensor, climate_reading) в PostgreSQL. Сложность в обеих моделях оценивается как $O(N_s \times \log T)$, где N_s — количество датчиков в корпусе, T — количество показаний на датчик. В InfluxDB требуется выполнение двух отдельных выборок с последующим соединением на уровне приложения, тогда как PostgreSQL позволяет выполнить всю логику одним запросом. При увеличении числа датчиков в

корпусе в 10 раз время выполнения возрастёт пропорционально в обеих моделях; при увеличении истории показаний на датчик — логарифмически.

Вывод по запросам:

- Для простых выборок по первичному ключу (список корпусов) PostgreSQL демонстрирует лучшую асимптотику благодаря B-tree индексам.
- Для агрегирующих запросов над временными рядами (график за 24 ч) InfluxDB эффективнее за счёт специализированной функции `aggregateWindow`, выполняющей агрегацию за линейное время.
- Для сценариев с соединением метаданных и показаний (датчики + последнее значение) обе модели показывают сопоставимую асимптотическую сложность, но PostgreSQL позволяет выполнить логику одним запросом через `LATERAL JOIN`, тогда как InfluxDB требует двух выборок с последующим соединением на уровне приложения.

2.3.3. Вывод — что лучше, SQL или NoSQL модель

По результатам сравнительного анализа:

Объём хранения. InfluxDB занимает в 2 раза меньше места на текущем наборе данных (1,60 МБ против 3,34 МБ) и до 18 раз меньше при прогнозе на год (≈ 65 МБ против $\approx 1,2$ ГБ) благодаря TSM-сжатию временных рядов.

Производительность чтения. PostgreSQL быстрее в простых выборках в 10 раз и в агрегирующих запросах — в 58 раз за счёт B-tree индексов и оптимизированного планировщика.

Разработка. InfluxDB упрощает фильтрацию по множеству атрибутов через автоматическую индексацию тегов; PostgreSQL требует явного управления индексами, но предлагает привычный SQL и мощные JOIN.

Масштабируемость. InfluxDB поддерживает горизонтальное масштабирование через шардинг по времени; PostgreSQL масштабируется преимущественно вертикально.

Итоговый вывод

Для системы мониторинга микроклимата, где:

- данные являются временными рядами с непрерывным потоком показаний,
 - критичен долгосрочный объём хранения,
 - допустима задержка чтения 10–60 мс,
- выбор InfluxDB (NoSQL) обоснован. Реляционная модель предпочтительнее для сценариев с приоритетом на минимальную задержку чтения и сложные аналитические запросы, но без требования долгосрочного хранения больших объёмов временных данных.

3. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

3.1. Краткое описание архитектуры

Приложение реализовано по клиент-серверной архитектуре и развёртывается с помощью Docker Compose. Система состоит из двух изолированных контейнеров:

- backend — серверная часть на Python 3.11 с фреймворком FastAPI и асинхронным сервером Uvicorn. Реализует REST API для CRUD-операций, аутентификации, импорта/экспорта данных и агрегирующих запросов. Также выступает в роли статического веб-сервера, отдающего файлы браузерного клиента.
- db — база данных InfluxDB 2.7. Контейнер не публикует порты во внешнюю сеть; доступ к СУБД осуществляется исключительно через внутреннюю Docker-сеть, что обеспечивает безопасность и изоляцию данных.

Браузерный клиент представляет собой одностраничное приложение (SPA), реализованное на нативном JavaScript без использования тяжёлых фреймворков. Навигация между разделами осуществляется динамически через переключение CSS-классов без перезагрузки страницы. Все данные хранятся исключительно в InfluxDB; mock-данные на фронтенде не используются. URL-адреса страниц однозначно кодируют состояние приложения (раздел, применённые фильтры, номер страницы пагинации), что соответствует требованиям к прототипу курса.

3.2. Используемые технологии

Стек технологий подобран с учётом требований к производительности, безопасности и воспроизводимости развёртывания. Основные компоненты приведены в таблице 4.

Таблица 4 — Технологический стек приложения

Компонент	Технология	Версия
Бэкенд	Python / FastAPI /	3.11 / 0.111 / 0.29

	Uvicorn	
СУБД	InfluxDB	2.7
Аутентификация	PyJWT	2.8
Валидация	Pydantic v2	2.x
Фронтенд — CSS	Bootstrap	5.3.3
Фронтенд — графики	Chart.js	4.4.3
Контейнеризация	Docker Compose	3.8
Язык запросов БД	Flux	—

3.3. Снимки экрана приложения

Интерфейс системы реализован в виде единого веб-приложения с боковым меню навигации. Основные экраны отражают ключевые сценарии использования: дашборд со сводной статистикой, таблицы объектов с многокритериальной фильтрацией и серверной пагинацией, модуль аналитики с настраиваемыми графиками и интерфейс импорта/экспорта данных. Ниже приведены снимки основных экранов приложения.

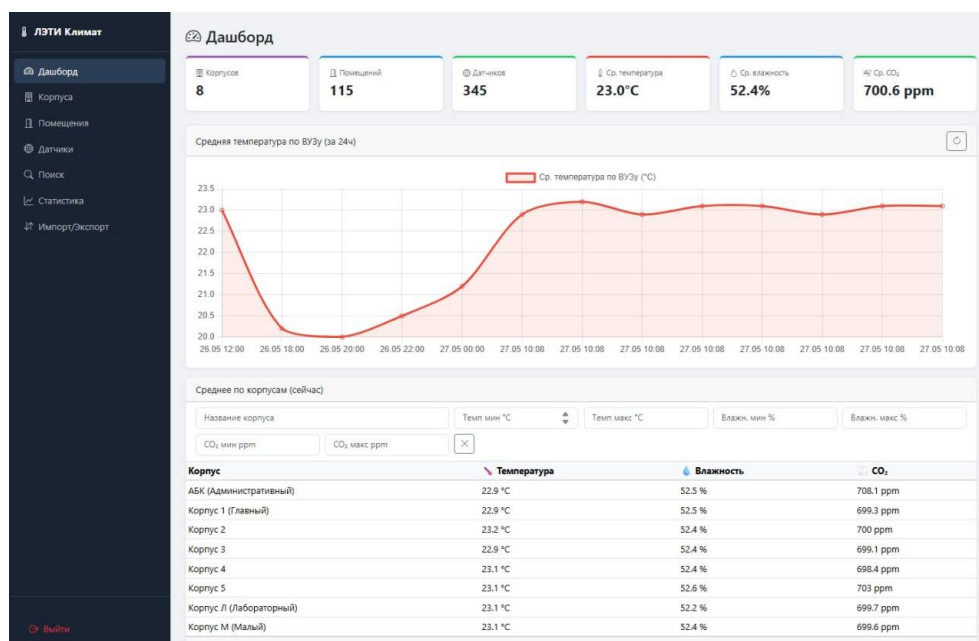


Рисунок 4 — Главный экран (Дашборд): сводные счётчики объектов, график средней температуры за 24 часа, таблица средних значений по корпусам фильтрами.

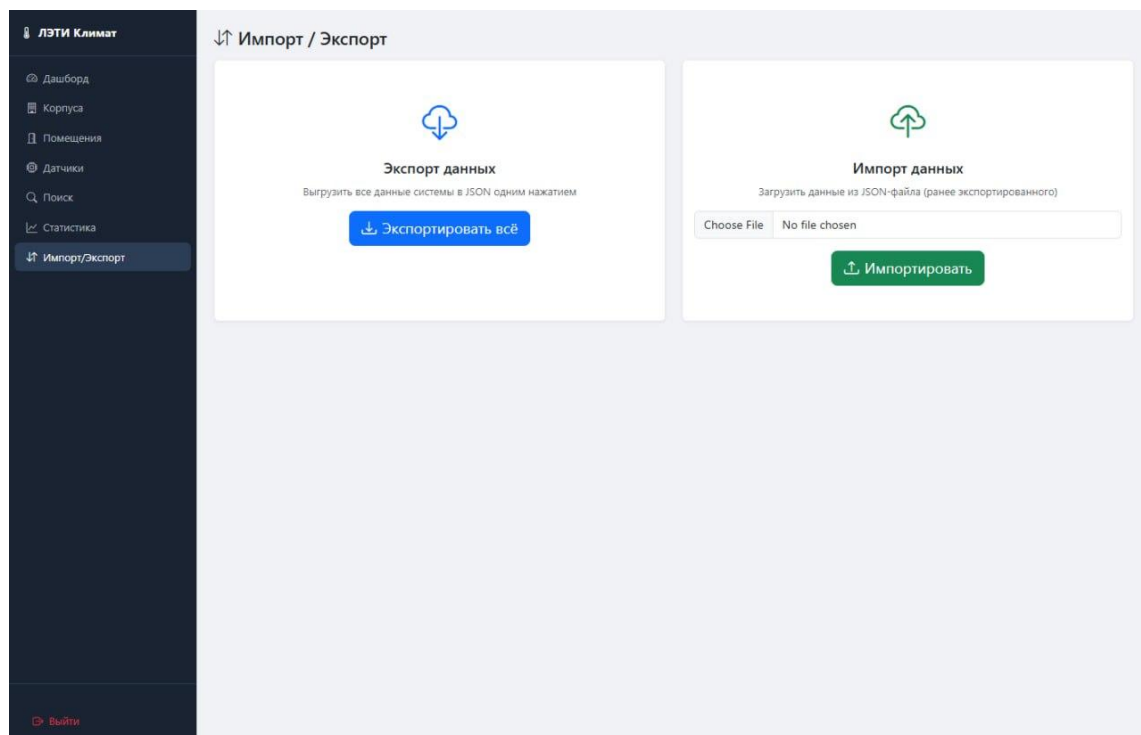


Рисунок 7 — Модуль импорта/экспорта: форма выгрузки всех данных в JSON и загрузки ранее экспортированного файла.

ЗАКЛЮЧЕНИЕ

Достигнутые результаты

1. Разработано и развёрнуто полнофункциональное веб-приложение для мониторинга микроклимата учебных корпусов СПбГЭТУ «ЛЭТИ».
2. Реализованы все обязательные сценарии использования: массовый импорт/экспорт данных, многокритериальная фильтрация, серверная пагинация, визуализация временных рядов и кастомизируемая аналитика.
3. Внедрена ролевая модель доступа (администратор/наблюдатель) с JWT-аутентификацией и строгой валидацией запросов на уровне API.
4. Проведён сравнительный анализ моделей данных InfluxDB и PostgreSQL: экспериментально подтверждены расчётные формулы объёма хранения и оценена производительность запросов на реальных данных.

Недостатки и пути улучшения

1. Отсутствует система оперативных оповещений при выходе метрик за пороговые значения.
Решение: внедрение модуля алертинга с настраиваемыми правилами и каналами уведомлений (Email, Telegram).
2. Экспорт данных выполняется без фильтрации по времени.
Решение: добавление параметров выбора периода в интерфейс и API экспорта.
3. Учётные записи пользователей хранятся в оперативной памяти приложения.
Решение: перенос данных аутентификации в персистентное хранилище (отдельная коллекция БД или SQLite).

4. Отсутствует графический интерфейс управления ролями.
Решение: разработка административной панели с функциями CRUD для учётных записей.

Будущее развитие решения

1. Интеграция с физическими IoT-датчиками через протоколы MQTT/HTTP для приёма телеметрии в реальном времени.
2. Внедрение продвинутой подсистемы алертинга с поддержкой гистерезиса, расписаний и многоканальных оповещений.
3. Подключение аналитической платформы Grafana для построения профессиональных дашбордов на базе данных InfluxDB.
4. Горизонтальное масштабирование серверной части через балансировщик нагрузки (NGINX/Traefik) и репликацию контейнеров.
5. Разработка мобильного клиента с поддержкой офлайн-режима и push-уведомлений о критических отклонениях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Репозиторий проекта nsq1h26-climate. — URL: <https://github.com/moevm/nsq1h26-climate>
2. InfluxDB 2.7 Documentation. — URL: <https://docs.influxdata.com/influxdb/v2/>
3. FastAPI Documentation. — URL: <https://fastapi.tiangolo.com/>
4. Методика расчёта объёма модели данных. — URL: http://se.moevm.info/doku.php/staff:courses:no_sql_introduction:calculating_data_model_size
5. Bootstrap 5 Documentation. — URL: <https://getbootstrap.com/docs/5.3/>
6. Chart.js Documentation. — URL: <https://www.chartjs.org/docs/>

ПРИЛОЖЕНИЕ А. ДОКУМЕНТАЦИЯ ПО СБОРКЕ И РАЗВЕРТЫВАНИЮ ПРИЛОЖЕНИЯ

Требования: Docker Desktop 24+ с включённым Docker Compose.

Команды для развёртывания:

```
# Клонировать репозиторий
git clone https://github.com/moevm/nsql1h26-climate.git
cd nsql1h26-climate

# Запустить приложение
docker compose -f leti_climate_prototype/docker-compose.yml up --
build -d

# Приложение доступно по адресу:
http://localhost:8080
```

Первый запуск занимает 15–30 секунд: InfluxDB инициализируется, затем seed-скрипт заполняет базу синтетическими данными.

ПРИЛОЖЕНИЕ Б. ИНСТРУКЦИЯ ДЛЯ ПОЛЬЗОВАТЕЛЯ

Авторизация: открыть <http://localhost:8080>, ввести логин и пароль.

Роль	Логин	Пароль	Возможности
Администратор	admin	admin123	Создание, редактирование, удаление объектов; импорт/экспорт
Наблюдатель	viewer	viewer123	Только просмотр данных

Основные сценарии:

1. Просмотр дашборда — автоматически после входа.
2. Фильтрация таблицы — заполнить поля фильтра, нажать «Применить».
3. Добавление корпуса — раздел «Корпуса» → кнопка «+».
4. График показаний — раздел «Статистика» → выбрать корпус, метрику, период.
5. Анализ распределения — «Статистика» → «Анализ распределения» → настроить фильтры и оси.
6. Экспорт — раздел «Импорт/Экспорт» → выбрать тип → «Экспортировать».